

Linear classification

Instructor: Roberto Colomboni

version of 27 September 2026

In Lecture 2, we used regression to predict numerical targets. We now turn to classification, where each target tells us which class an observation belongs to. Our aim is therefore to choose a classifier that predicts the correct label as often as possible.

We will develop the theory for inputs with any number d of features, using examples with one or two features whenever a picture or a short calculation helps explain the general construction. We begin with two classes and build a model that predicts a probability before choosing a label. This gives us more information than the label alone, which will be useful when fitting the model.

Once we have defined the classifier, we will ask how to choose its parameters from data. Counting classification mistakes gives an objective that stays flat whenever the predicted labels stay the same, so its gradient does not tell us how to improve the fit. We will therefore introduce a smooth *surrogate* loss—the *logistic* loss—explain its connection to the number of mistakes, and derive the gradient-descent updates. Finally, we will extend the same ideas to more than two classes.

1 Binary classification: predictions, probabilities and decision boundaries

1.1 Our objective: predict the correct class

Suppose that we have $N \geq 1$ observations, each with $d \geq 1$ input features:

$$(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N), \quad \mathbf{x}_i \in \mathbb{R}^{1 \times d}, \quad y_i \in \{0, 1\}.$$

As in Lecture 2, we represent an input by a row. In particular, writing $\mathbf{x}_i = (x_{i1}, \dots, x_{id})$, the first subscript identifies an observation, while the second identifies one of its features. The target y_i records the observed class, so the values 0 and 1 are names for the two groups rather than numerical targets that we want to approximate.

Because there are two possible labels, we call this task *binary classification*. A *classifier* is a function

$$h : \mathbb{R}^d \rightarrow \{0, 1\},$$

where we continue to write its inputs as rows. Thus $h(\mathbf{x})$ is the label predicted by the classifier h at input $\mathbf{x}$. To judge this prediction, we compare it with the observed label. If all mistakes have the same cost, it is sensible to use the *zero-one loss*

$$\ell_{0,1}(\hat{y}, y) := \mathbf{1}\{\hat{y} \neq y\} = \begin{cases} 0, & \hat{y} = y, \\ 1, & \hat{y} \neq y. \end{cases} \quad (1)$$

Adding these losses over the dataset gives the number of mistakes,

$$\sum_{i=1}^N \ell_{0,1}(h(\mathbf{x}_i), y_i).$$

Consequently, dividing this sum by N gives the error rate, while subtracting that rate from one gives the accuracy. Notice that minimising the number of mistakes is equivalent to maximising accuracy on these observations.

Ultimately, however, we want to predict labels correctly on unseen data. As in regression, fitting the observed data and generalising to new data are different questions, and phenomena such as *underfitting* and *overfitting* can prevent a model from generalising well.

In this lecture, we will again concentrate on the fitting calculation. Throughout fitting, the inputs and observed labels stay fixed, while the parameters of our model change.

1.2 Predict probabilities, then choose a label

A classifier ultimately has to output a class label. However, rather than producing this label directly, we will construct the prediction in two steps. First, for each input, the model will assign a probability $p \in [0, 1]$ to class 1, and therefore probability $1 - p$ to class 0. We then predict the class with the larger probability: class 1 if $p \geq 1/2$, and class 0 otherwise. In the case of a tie, we adopt the somewhat arbitrary convention of choosing class 1.

The probability contains more information than the final label alone. For example, if we are required to make a class prediction, $p = 0.51$ still leads us to choose class 1, albeit perhaps only reluctantly, whereas $p = 0.99$ lets us make the same prediction with much greater confidence.

This distinction will be useful when fitting the model. Rather than considering only whether the final class prediction is correct, we will introduce a surrogate loss that also takes into account how confident the model is in that prediction, penalising high-confidence mistakes more heavily.

For now, suppose that we have a function

$$p : \mathbb{R}^d \rightarrow [0, 1].$$

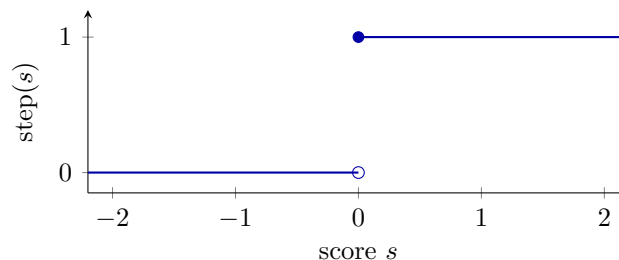
We interpret $p(\mathbf{x})$ as the probability that the model assigns to class 1 when the input is $\mathbf{x}$, so class 0 receives probability $1 - p(\mathbf{x})$. The *plug-in classifier* associated to the function p chooses whichever label has the larger predicted probability:

$$h_p(\mathbf{x}) := \begin{cases} 0, & p(\mathbf{x}) < 1/2, \\ 1, & p(\mathbf{x}) \geq 1/2. \end{cases} \quad (2)$$

Thus we predict class 1 when its probability is at least $1/2$, including a tie. To write this rule more compactly, we define the *Heaviside step function*

$$\text{step}(s) := \begin{cases} 0, & s < 0, \\ 1, & s \geq 0. \end{cases} \quad (3)$$

Its graph shows the jump at zero, where our convention gives $\text{step}(0) = 1$.



It follows that the same plug-in classifier can be written as

$$h_p(\mathbf{x}) = \text{step}\left(p(\mathbf{x}) - \frac{1}{2}\right).$$

1.3 From an affine score to a probability

The plug-in rule tells us how to use a probability prediction, but we still need to choose the function that produces it. To do so, we can try to imitate what we did in linear regression and see what happens if we use an affine function of the input. However, an affine function can take values outside $[0, 1]$, so we cannot use it directly as a probability. Instead, we will treat this *score* as an intermediate quantity, and then transform it into a number between zero and one.

As in affine regression, we give each feature its own weight and include an intercept. We collect these parameters in a column:

$$\boldsymbol{\theta} = \begin{pmatrix} b \\ \mathbf{w} \end{pmatrix} = \begin{pmatrix} b \\ w_1 \\ \vdots \\ w_d \end{pmatrix} \in \mathbb{R}^{(d+1) \times 1}, \quad \mathbf{w} \in \mathbb{R}^{d \times 1}, \quad b \in \mathbb{R}. \quad (4)$$

For an input row

$$\mathbf{x} = (x_1, \dots, x_d) \in \mathbb{R}^{1 \times d},$$

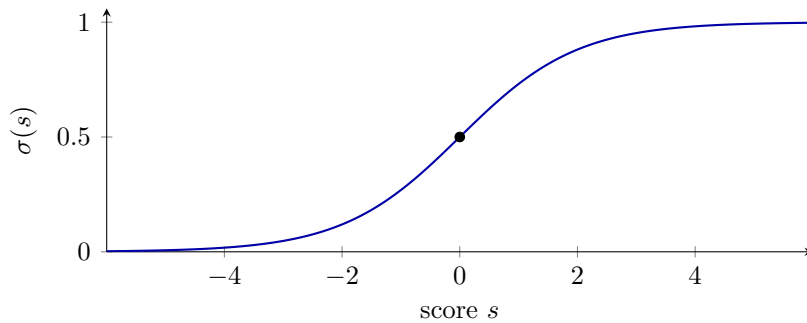
we define the *score* associated with $\boldsymbol{\theta}$ by

$$f_{\boldsymbol{\theta}}(\mathbf{x}) := b + \mathbf{x}\mathbf{w} = b + \sum_{j=1}^d x_j w_j. \quad (5)$$

To turn the score into a probability, we use the *logistic sigmoid*

$$\sigma : \mathbb{R} \rightarrow (0, 1), \quad \sigma(s) := \frac{1}{1 + e^{-s}} = \frac{e^s}{1 + e^s}. \quad (6)$$

Because $e^{-s} > 0$, every score gives a value strictly between zero and one. Moreover, the sigmoid is strictly increasing and satisfies $\sigma(0) = 1/2$. A large positive score therefore gives a probability close to one, while a large negative score gives a probability close to zero.



Composing the score with the sigmoid gives our probability model:

$$\boxed{p_{\boldsymbol{\theta}}(\mathbf{x}) := \sigma(f_{\boldsymbol{\theta}}(\mathbf{x})) = \sigma(b + \mathbf{x}\mathbf{w})}. \quad (7)$$

Other transformations into $(0, 1)$ are possible, so choosing the sigmoid to turn scores into probabilities is a modelling choice. We use the sigmoid because it works particularly well with the smooth surrogate loss—the logistic loss—that we will introduce below. In fact, when we differentiate the resulting loss, the expression will simplify to a probability-minus-label difference. The score is also called a *logit*. Its connection with log-odds is explained in Appendix B.

1.4 The decision boundary is a hyperplane

Since the sigmoid is strictly increasing and $\sigma(0) = 1/2$, we have

$$\sigma(s) \geq \frac{1}{2} \iff s \geq 0.$$

Therefore, applying the plug-in rule to our model gives

$$\begin{aligned} h_{p_\theta}(\mathbf{x}) &= \text{step}\left(p_\theta(\mathbf{x}) - \frac{1}{2}\right) = \text{step}\left(\sigma(f_\theta(\mathbf{x})) - \frac{1}{2}\right) = \text{step}(f_\theta(\mathbf{x})) = \text{step}(b + \mathbf{x}\mathbf{w}) \\ &= \begin{cases} 0, & b + \mathbf{x}\mathbf{w} < 0, \\ 1, & b + \mathbf{x}\mathbf{w} \geq 0. \end{cases} \end{aligned} \quad (8)$$

We see then that the predicted label consequently depends only on the sign of the score. On the other hand, the sigmoid adds a smooth probability prediction, which we will use when fitting the parameters.

When $\mathbf{w} \neq \mathbf{0}$, the equation

$$b + \mathbf{x}\mathbf{w} = 0 \quad (9)$$

defines an affine *hyperplane* in the input space $\mathbb{R}^d$.¹ This hyperplane is called the *decision boundary* of the classifier, because it separates the region where the score is negative, and hence the prediction is class 0, from the region where the score is positive, and hence the prediction is class 1. Notice that, with our convention for breaking ties, points on the hyperplane itself receive class 1.

The intercept allows the hyperplane to move away from the origin, whereas setting $b = 0$ forces it to pass through the origin. The only exceptional case is when $\mathbf{w} = \mathbf{0}$: in this case, the score is constant, so we predict the same label everywhere and there is no separating hyperplane.

A classifier constructed in this way is called a *linear classifier*. The term *linear* refers to the affine score used to make the decision and, in particular, does not mean that the predicted probability $p_\theta(\mathbf{x})$ is a linear function of the input.

Example with one input feature: a threshold. When $d = 1$, we write the single input as x and its weight as $a = w_1$. The general score then becomes

$$f_\theta(x) = b + ax.$$

If $a \neq 0$, the decision boundary is the single point $-b/a$. In particular, when $a > 0$,

$$h_{p_\theta}(x) = 1 \iff b + ax \geq 0 \iff x \geq -\frac{b}{a}. \quad (10)$$

¹In one dimension, a hyperplane is simply a point; in two dimensions, it is a straight line; and in three dimensions, it is a plane.

Thus the classifier is a *threshold rule* on the input axis. If $a < 0$, the inequality reverses, so class 1 is predicted to the left of the threshold, including the threshold itself. If $a = 0$, the classifier is constant, as in the general case discussed above.

Figure 1 shows the choice $a = 1$ and $b = -1$. In this example, predicting class 1 when its probability is at least $1/2$ is equivalent to predicting class 1 when $x \geq 1$.

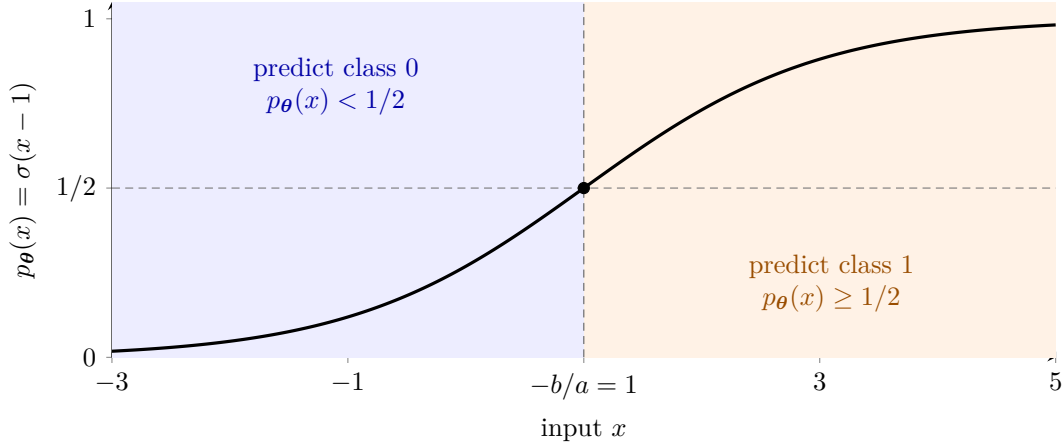


Figure 1: With one input feature, a linear classifier is a threshold rule. Here $f_{\theta}(x) = x - 1$: the classifier predicts 0 for $x < 1$ and 1 for $x \geq 1$. The curve shows the predicted probability, while the shaded regions show the labels selected by the plug-in rule. At $x = 1$, the score is zero and the predicted probability is $1/2$.

Example with two input features: a line. When $d = 2$, an input

$$\mathbf{x} = (x_1, x_2)$$

is a point in the (x_1, x_2) -plane. In this case, the decision-boundary equation (9) becomes

$$w_1 x_1 + w_2 x_2 + b = 0. \quad (11)$$

For $\mathbf{w} \neq \mathbf{0}$, this is the equation of a straight line. The line divides the plane into two regions: on one side the score is negative and we predict class 0, while on the other side the score is positive and we predict class 1.

For example, consider

$$\boldsymbol{\theta} = (-1, -0.6, 1)^{\top}.$$

The corresponding score is

$$f_{\theta}(\mathbf{x}) = x_2 - 0.6x_1 - 1,$$

so

$$h_{p_{\theta}}(\mathbf{x}) = 1 \iff f_{\theta}(\mathbf{x}) \geq 0 \iff x_2 \geq 0.6x_1 + 1. \quad (12)$$

Therefore the decision boundary is the line

$$x_2 = 0.6x_1 + 1.$$

The classifier predicts class 1 on and above this line and class 0 below it, as shown in Figure 2.

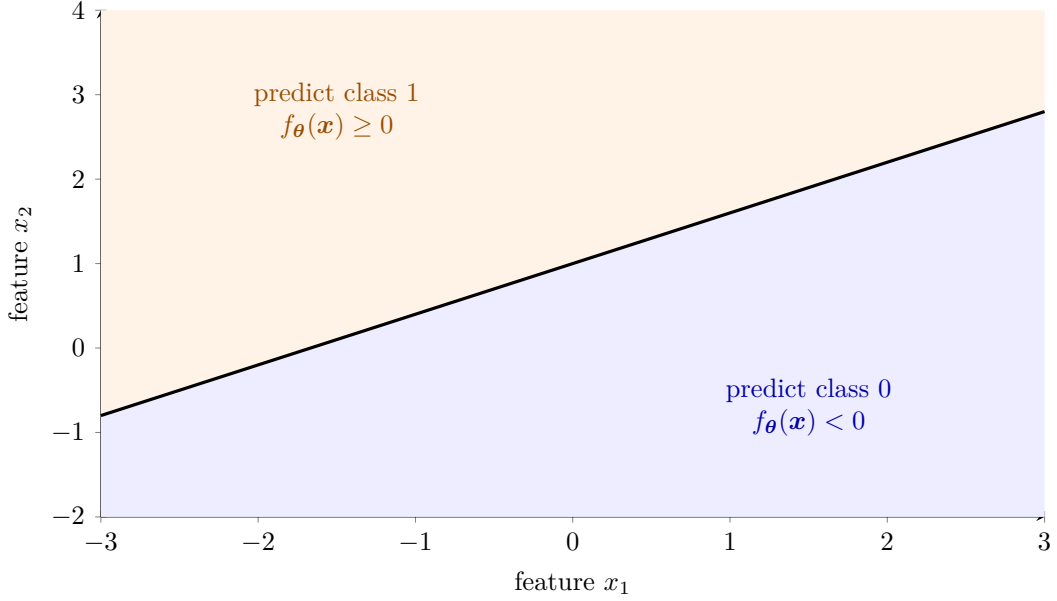


Figure 2: A linear classifier with two input features. The black line is the decision boundary $x_2 = 0.6x_1 + 1$. The shaded regions show the labels predicted by the classifier: class 0 below the line and class 1 on and above it.

2 From classification error to the logistic loss

2.1 The empirical zero-one loss

Having chosen a family of classifiers, we can now ask how to choose its parameters from the observed data. Since our original aim is to predict the correct class as often as possible, a natural objective is simply to count the number of mistakes.

We therefore define the *empirical zero-one loss*

$$\mathcal{E}(\boldsymbol{\theta}) := \sum_{i=1}^N \ell_{0,1}(h_{p_{\boldsymbol{\theta}}}(\mathbf{x}_i), y_i) = \sum_{i=1}^N \mathbf{1}\{h_{p_{\boldsymbol{\theta}}}(\mathbf{x}_i) \neq y_i\}. \quad (13)$$

Thus $\mathcal{E}(\boldsymbol{\theta})$ is simply the number of observations that are misclassified by the classifier with parameters $\boldsymbol{\theta}$.

Example with two input features. To make this concrete, consider a dataset with two input features. Each observation can be represented by a point in the (x_1, x_2) -plane, while its colour and symbol indicate its observed class.

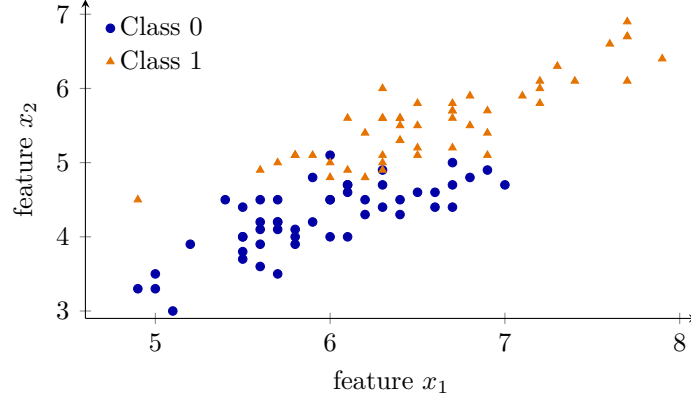


Figure 3: A dataset with two input features. Each point represents one observation in the (x_1, x_2) -plane, while its colour and symbol indicate its observed class.

Comparing two boundaries on the same observations. Consider the two parameter choices

$$\begin{aligned}\theta_A &= (-5.75, 0, 1)^\top, & f_{\theta_A}(\mathbf{x}) &= x_2 - 5.75, \\ \theta_B &= (-1, -0.6, 1)^\top, & f_{\theta_B}(\mathbf{x}) &= x_2 - 0.6x_1 - 1.\end{aligned}\tag{14}$$

Their decision boundaries are respectively

$$x_2 = 5.75 \quad \text{and} \quad x_2 = 0.6x_1 + 1.$$

The second one is the classifier that we used in the previous section to illustrate a decision boundary with two input features. Figure 4 now shows both classifiers on the same observed data.

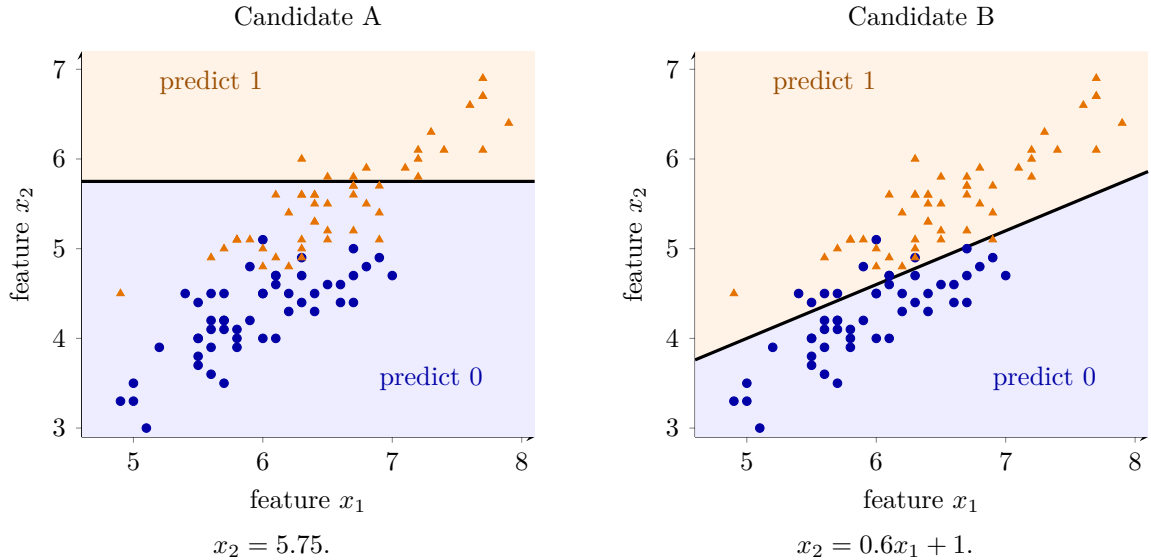


Figure 4: Two candidate linear classifiers on exactly the same observations and with identical axis scales. Marker colours and shapes show the observed labels, while the shaded regions show the predicted labels.

Thus the empirical zero-one loss distinguishes the two parameter choices. From Figure 4, we can see that Candidate B makes fewer mistakes than Candidate A, and hence

$$\mathcal{E}(\theta_B) < \mathcal{E}(\theta_A).$$

If our aim is to make as few mistakes as possible, we would therefore like to choose the parameters by minimising $\mathcal{E}(\boldsymbol{\theta})$.

Unfortunately, unlike least squares in linear regression, this optimisation problem does not lead to a comparable closed-form solution. A direct search over the possible classification patterns is, in general, a combinatorial problem and can quickly become computationally prohibitive as the size of the problem grows.

In Lecture 2, however, we saw that gradient descent provides an iterative way of approaching a minimiser, without having to compute it explicitly. Can we use the same idea here?

2.2 Why counting mistakes does not guide gradient descent

Unfortunately, the way this objective depends on the parameters is quite different from the squared loss.

Consider, for example, a parameter choice for which every observed score is nonzero, i.e., every observation lies strictly on one side of the decision boundary. Because each score is an affine function of the parameters, it depends continuously on them. Therefore, if none of the observed scores is zero, a sufficiently small change in the parameters keeps all the scores on the same side of zero. Consequently, there exists a small neighbourhood of this parameter choice in which every predicted label stays the same, and therefore so does the number of mistakes.

Hence $\mathcal{E}$ is locally constant around this parameter choice. Since the gradient of a locally constant function is zero, we have

$$\nabla \mathcal{E}(\boldsymbol{\theta}) = \mathbf{0} \quad \text{whenever every observed score is nonzero.}$$

Thus the gradient provides no direction in which to move towards a region with fewer mistakes.

2.3 Example: the error count for a threshold

The preceding argument holds for any number of input features. To see its effect in a picture, we now take $d = 1$ and write $a = w_1$. Within this example, each input is a single number, so we write it as x_i , while the parameter column is $\boldsymbol{\theta} = (b, a)^\top$. We use the following seven observations:

i	1	2	3	4	5	6	7	
x_i	-2	-1	0	1	2	3	4	
y_i	0	0	0	1	1	0	1	(15)

Most larger inputs have label 1, but the point at $x = 3$ has label 0. Therefore, the two classes are not separated perfectly by a single threshold on the input axis. This lets us illustrate the fitting difficulty with a graph.

Before examining the gradient, let us check what a threshold can achieve on these data. The score $x - 1/2$ predicts class 1 exactly when $x \geq 1/2$, so it correctly labels every point except $x = 3$. It therefore makes one mistake. The observed labels, read from left to right as the input increases, are

$$0, 0, 0, 1, 1, 0, 1.$$

As we move from left to right, these labels change three times. By contrast, a threshold classifier can change its predicted label at most once: it predicts one class on one side of the threshold and the other class on the other side. Therefore, no threshold can classify all seven observations correctly. Since the threshold $x \geq 1/2$ shown above makes only one mistake, one mistake is the best possible result in this family.

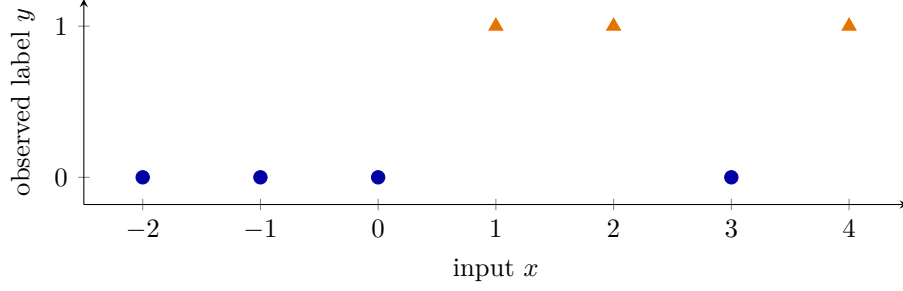


Figure 5: Each observation has one input feature. The vertical coordinate shows the label, so it does not represent a second input feature.

In one dimension, finding the best threshold is straightforward: after sorting the inputs, we can try every possible threshold position between two consecutive observations, with either class assigned to the left-hand side, as well as the two constant classifiers. While this provides a direct solution in this special case, we run into trouble if we try to find a minimum using gradient descent. Indeed, consider the parameter choice $\theta = (3, 1)^\top$. The resulting scores on the seven observations are 1, 2, 3, 4, 5, 6, 7, so every score is positive and the classifier makes four mistakes. Since all seven scores are nonzero, the error count is locally constant around $\theta = (3, 1)^\top$, and hence

$$\nabla \mathcal{E}((3, 1)^\top) = \mathbf{0}.$$

Therefore, a gradient-descent update leaves the parameters unchanged:

$$\begin{pmatrix} 3 \\ 1 \end{pmatrix} - \eta \mathbf{0} = \begin{pmatrix} 3 \\ 1 \end{pmatrix}.$$

This happens for every learning rate $\eta > 0$, even though we already know that $(-1/2, 1)^\top$ gives only one mistake.

We can also visualise these flat regions by fixing $a = 1$ and varying only b (see Figure 6). The predicted label at input x_i changes when $b = -x_i$, which gives the following error count:

$$\mathcal{E}((b, 1)^\top) = \begin{cases} 3, & b < -4, \\ 2, & -4 \leq b < -3, \\ 3, & -3 \leq b < -2, \\ 2, & -2 \leq b < -1, \\ 1, & -1 \leq b < 0, \\ 2, & 0 \leq b < 1, \\ 3, & 1 \leq b < 2, \\ 4, & 2 \leq b. \end{cases} \quad (16)$$

At each breakpoint, the score of one observation is zero, so our tie rule predicts class 1 for that observation. This explains which endpoints belong to each interval in the formula.

We have therefore reached a difficulty: the error count measures exactly what we want to minimise, but we do not have a practical way to minimise it in general, and gradient descent does not seem to help.

On the other hand, if we do not want to give up on gradient descent, we might instead try to construct a different objective to use during fitting.

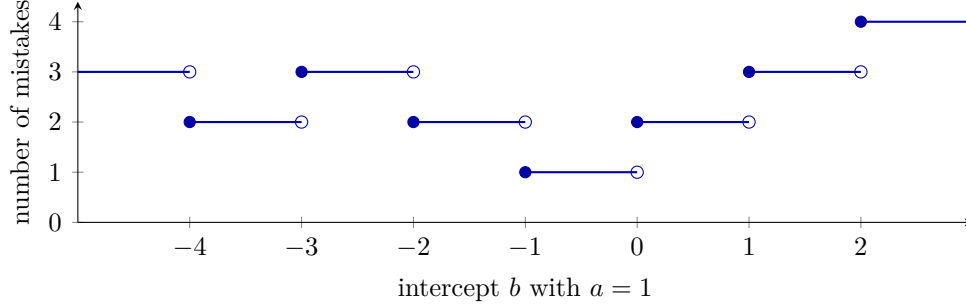


Figure 6: Changing the intercept moves the threshold, but the error count changes only when an observed input crosses that threshold. Filled points show the value at a breakpoint, while open points are excluded.

This leads to the following question: can we construct an objective that changes smoothly as the parameters change, while still retaining a meaningful connection with the number of classification mistakes that we ultimately want to minimise?

2.4 A smooth surrogate: the logistic loss

One possible way to construct such an objective is to use the probabilities that our model already produces. These probabilities contain information that is discarded by the plug-in rule and, consequently, by the error count.

For example, if the observed label is 1, increasing its predicted probability from 0.6 to 0.9 does not change the predicted class, but it does make the prediction more confident in class 1, which is the class that was actually observed. In this case, we would therefore like the fitting loss to decrease as this probability increases.

Probabilities can also tell us how strongly to penalise a mistake. For example, suppose that the observed label is 0 but the model assigns probability 0.51 to class 1. The plug-in rule forces the classifier to choose class 1, so this counts as a mistake, even though the model was almost undecided between the two classes. We would therefore not want to penalise this mistake as strongly as one in which the model assigns probability 0.99 to the wrong class.

A simple way to achieve these properties is to take the negative logarithm of the probability assigned to the observed label. If $y = 1$, that probability is p , while if $y = 0$, it is $1 - p$. This leads to the *logistic loss*, also called *binary cross-entropy*:

$$\ell_{\log}(p, y) := -y \log p - (1 - y) \log(1 - p) = \begin{cases} -\log p, & \text{if } y = 1, \\ -\log(1 - p), & \text{if } y = 0, \end{cases} \quad (17)$$

where $p \in (0, 1)$, $y \in \{0, 1\}$, and we recall that $\log$ denotes the natural logarithm.

Since the negative logarithm decreases as its argument increases, assigning a larger probability to the observed label gives a smaller loss. Conversely, a prediction that assigns almost no probability to the observed label receives a large loss.

For example, suppose again that $y = 0$. If the model assigns probability $p = 0.51$ to class 1, then the probability assigned to the observed class is 0.49, and the loss is

$$-\log(0.49) \approx 0.713.$$

If instead the model assigns probability $p = 0.99$ to class 1, the probability assigned to the observed class is only 0.01, and the loss becomes

$$-\log(0.01) \approx 4.605.$$

Thus both predictions are mistakes, but the logistic loss penalises the highly confident mistake much more strongly than the almost undecided one.

So far, we have defined a loss for a single observation. As we did for the other losses in Lecture 2, we obtain an objective for fitting the parameters of our model by adding the losses over all the observations in the dataset. This gives the *empirical logistic loss*

$$\mathcal{L}_{\log}(\boldsymbol{\theta}) := \sum_{i=1}^N \ell_{\log}(p_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i). \quad (18)$$

As in Lecture 2, the lowercase ℓ refers to the loss of one observation, whereas $\mathcal{L}$ refers to the sum over the dataset.

We now have a candidate objective for fitting the model. This objective is smooth as a function of the parameters: the score is an affine function of $\boldsymbol{\theta}$, the sigmoid is smooth, and the logarithm is smooth on $(0, 1)$, where the sigmoid takes its values. A finite sum of these individual losses is therefore also smooth.

However, smoothness alone is not enough to justify our choice. We now need to check that this loss is connected to the classification mistakes $\mathcal{E}$ that we actually care about.

2.5 Why a small logistic loss implies few mistakes

The connection follows from one observation: whenever the classifier makes a mistake, the probability assigned to the observed class is at most $1/2$. Therefore, each incorrect prediction contributes at least $-\log(1/2) = \log 2$ to the logistic loss.

Indeed, if $y = 1$, a mistake means that $p < 1/2$, so

$$\ell_{\log}(p, 1) = -\log p > \log 2.$$

If instead $y = 0$, a mistake means that $p \geq 1/2$. It follows that $1 - p \leq 1/2$, and hence

$$\ell_{\log}(p, 0) = -\log(1 - p) \geq \log 2.$$

On the other hand, when the prediction is correct, the zero-one loss is zero and the logistic loss is nonnegative.

Consequently, if $p : \mathbb{R}^d \rightarrow (0, 1)$ is a probability predictor and h_p is its associated plug-in classifier, then for every input $\mathbf{x}$ and label $y \in \{0, 1\}$,

$$(\log 2) \ell_{0,1}(h_p(\mathbf{x}), y) \leq \ell_{\log}(p(\mathbf{x}), y). \quad (19)$$

Equivalently,

$$\boxed{\ell_{0,1}(h_p(\mathbf{x}), y) \leq \frac{\ell_{\log}(p(\mathbf{x}), y)}{\log 2}.} \quad (20)$$

We can also visualise this relation as a function of the score. For our sigmoid model, at a fixed input the probability has the form

$$p = \sigma(s) = \frac{1}{1 + e^{-s}}.$$

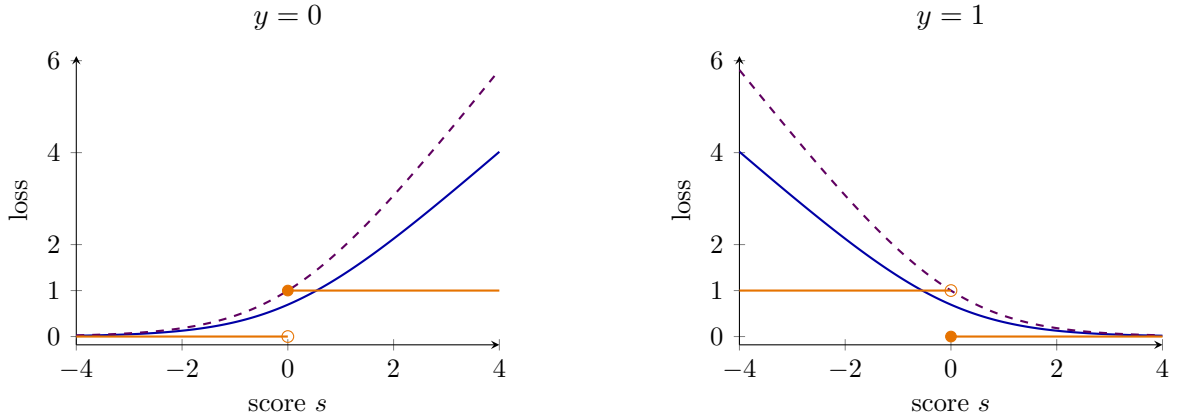
Moreover,

$$1 - \sigma(s) = 1 - \frac{1}{1 + e^{-s}} = \frac{e^{-s}}{1 + e^{-s}} = \frac{1}{1 + e^s} = \sigma(-s). \quad (21)$$

Substituting these expressions into the logistic loss gives

$$\begin{aligned} \ell_{\log}(\sigma(s), 1) &= -\log\left(\frac{1}{1 + e^{-s}}\right) = \log(1 + e^{-s}), \\ \ell_{\log}(\sigma(s), 0) &= -\log\left(\frac{1}{1 + e^s}\right) = \log(1 + e^s). \end{aligned} \quad (22)$$

Since $\sigma(s) \geq 1/2$ exactly when $s \geq 0$, the corresponding plug-in prediction is $\text{step}(s)$. The following plots therefore show both sides of (20) as the score varies.



Blue: the logistic loss $\ell_{\log}(\sigma(s), y)$. Dashed violet: the rescaled logistic loss $\ell_{\log}(\sigma(s), y)/\log 2$. Orange: the zero-one loss $\ell_{0,1}(\text{step}(s), y)$. The zero-one loss changes abruptly when the predicted label changes, whereas the logistic loss varies smoothly. The dashed curve lies above the zero-one loss, as guaranteed by (20). Filled points show the convention $\text{step}(0) = 1$ from (3).

We can now apply the pointwise relation to our model by taking $p = p_{\theta}$. For every observation i ,

$$\ell_{0,1}(h_{p_{\theta}}(\mathbf{x}_i), y_i) \leq \frac{\ell_{\log}(p_{\theta}(\mathbf{x}_i), y_i)}{\log 2}.$$

Adding these inequalities over all the observations gives

$$\boxed{\mathcal{E}(\theta) \leq \frac{\mathcal{L}_{\log}(\theta)}{\log 2}}. \quad (23)$$

This gives a direct interpretation of the logistic loss. For example, if, for some positive integer n ,

$$\mathcal{L}_{\log}(\theta) < n \log 2,$$

then $\mathcal{E}(\theta) < n$. Since the number of mistakes is an integer, the classifier can make at most $n - 1$ mistakes.

In particular, if

$$\mathcal{L}_{\log}(\theta) < \log 2,$$

then $\mathcal{E}(\theta) < 1$, and hence the classifier makes no mistakes on the dataset.

We have therefore established the connection with the error count that motivated the surrogate. We now turn to the derivatives needed to fit the parameters using gradient descent.

2.6 Writing all observations with one design matrix

Before differentiating the loss, it is convenient to organise the inputs and parameters exactly as we did in Lecture 2.

Since the score contains an intercept, we place a column of ones before the input features and define the *design matrix*

$$\mathbf{X} = \begin{pmatrix} 1 & \mathbf{x}_1 \\ \vdots & \vdots \\ 1 & \mathbf{x}_N \end{pmatrix} = \begin{pmatrix} 1 & x_{11} & \cdots & x_{1d} \\ \vdots & \vdots & & \vdots \\ 1 & x_{N1} & \cdots & x_{Nd} \end{pmatrix} \in \mathbb{R}^{N \times (d+1)}. \quad (24)$$

The corresponding parameter column is

$$\boldsymbol{\theta} = \begin{pmatrix} b \\ w_1 \\ \vdots \\ w_d \end{pmatrix} \in \mathbb{R}^{(d+1) \times 1}.$$

Thus $\theta_1 = b$, while $\theta_{j+1} = w_j$ for $j = 1, \dots, d$. Similarly, the first column of $\mathbf{X}$ contains ones, while column $j + 1$ contains the values of feature j .

For observation i , multiplying row i of the design matrix by the parameter column gives

$$(\mathbf{X}\boldsymbol{\theta})_i = \sum_{r=1}^{d+1} X_{ir}\theta_r = b + \mathbf{x}_i\mathbf{w} = f_{\boldsymbol{\theta}}(\mathbf{x}_i). \quad (25)$$

Therefore, using the Kronecker delta, for every parameter θ_j ,

$$\frac{\partial f_{\boldsymbol{\theta}}(\mathbf{x}_i)}{\partial \theta_j} = \frac{\partial}{\partial \theta_j} \sum_{r=1}^{d+1} X_{ir}\theta_r = \sum_{r=1}^{d+1} X_{ir} \frac{\partial \theta_r}{\partial \theta_j} = \sum_{r=1}^{d+1} X_{ir} \delta_{rj} = X_{ij}, \quad j = 1, \dots, d+1, \quad (26)$$

This single formula includes the intercept, for which $X_{i1} = 1$, as well as all the feature weights.

For the whole dataset, we can express the column containing all the scores by

$$\begin{pmatrix} f_{\boldsymbol{\theta}}(\mathbf{x}_1) \\ \vdots \\ f_{\boldsymbol{\theta}}(\mathbf{x}_N) \end{pmatrix} = \mathbf{X}\boldsymbol{\theta} \in \mathbb{R}^{N \times 1}. \quad (27)$$

2.7 The chain rule for one observation

We can now differentiate the loss contributed by one observation. For a fixed pair $(\mathbf{x}_i, y_i)$, observe that it holds

$$\ell_{\log}(p_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i) = \ell_{\log}(\sigma(f_{\boldsymbol{\theta}}(\mathbf{x}_i)), y_i).$$

Hence, for any parameter θ_j , with $j = 1, \dots, d+1$, the chain rule gives

$$\frac{\partial}{\partial \theta_j} \ell_{\log}(p_{\boldsymbol{\theta}}(\mathbf{x}_i), y_i) = \frac{\partial}{\partial \theta_j} \ell_{\log}(\sigma(f_{\boldsymbol{\theta}}(\mathbf{x}_i)), y_i) = \frac{\partial \ell_{\log}}{\partial p}(\sigma(f_{\boldsymbol{\theta}}(\mathbf{x}_i)), y_i) \sigma'(f_{\boldsymbol{\theta}}(\mathbf{x}_i)) \frac{\partial f_{\boldsymbol{\theta}}(\mathbf{x}_i)}{\partial \theta_j}. \quad (28)$$

We therefore need to compute the three factors in this product.

First, differentiate the logistic loss with respect to the probability. For $p \in (0, 1)$ and $y \in \{0, 1\}$, we have

$$\frac{\partial \ell_{\log}}{\partial p}(p, y) = -\frac{y}{p} + \frac{1-y}{1-p} = \frac{-y(1-p) + (1-y)p}{p(1-p)} = \frac{p-y}{p(1-p)}. \quad (29)$$

Second, differentiate the probability with respect to the score. Since

$$\sigma(s) = \frac{1}{1 + e^{-s}},$$

we have

$$\sigma'(s) = -(1 + e^{-s})^{-2}(-e^{-s}) = \frac{e^{-s}}{(1 + e^{-s})^2} = \frac{1}{1 + e^{-s}} \frac{e^{-s}}{1 + e^{-s}} = \sigma(s)(1 - \sigma(s)). \quad (30)$$

Substituting $s = f_{\theta}(\mathbf{x}_i)$ therefore gives

$$\sigma'(f_{\theta}(\mathbf{x}_i)) = \sigma(f_{\theta}(\mathbf{x}_i))(1 - \sigma(f_{\theta}(\mathbf{x}_i))) = p_{\theta}(\mathbf{x}_i)(1 - p_{\theta}(\mathbf{x}_i)). \quad (31)$$

Third, differentiate the score with respect to the parameter. This is exactly the derivative that we obtained from the design matrix in (26):

$$\frac{\partial f_{\theta}(\mathbf{x}_i)}{\partial \theta_j} = X_{ij}.$$

Putting the three derivatives together. Substituting the three factors into (28) gives

$$\frac{\partial}{\partial \theta_j} \ell_{\log}(p_{\theta}(\mathbf{x}_i), y_i) = \frac{p_{\theta}(\mathbf{x}_i) - y_i}{p_{\theta}(\mathbf{x}_i)(1 - p_{\theta}(\mathbf{x}_i))} \cdot p_{\theta}(\mathbf{x}_i)(1 - p_{\theta}(\mathbf{x}_i)) \cdot X_{ij} = (p_{\theta}(\mathbf{x}_i) - y_i)X_{ij}.$$

The cancellation is valid because $0 < p_{\theta}(\mathbf{x}_i) < 1$ for every finite parameter choice. Hence

$$\boxed{\frac{\partial}{\partial \theta_j} \ell_{\log}(p_{\theta}(\mathbf{x}_i), y_i) = (p_{\theta}(\mathbf{x}_i) - y_i)X_{ij}, \quad j = 1, \dots, d+1.} \quad (32)$$

2.8 The gradient of the empirical objective

The empirical logistic loss in (18) adds the losses of all N observations. Differentiating this finite sum term by term and using (32) gives

$$\frac{\partial \mathcal{L}_{\log}}{\partial \theta_j}(\boldsymbol{\theta}) = \sum_{i=1}^N X_{ij}(p_{\theta}(\mathbf{x}_i) - y_i), \quad j = 1, \dots, d+1. \quad (33)$$

These derivatives are precisely the coordinates of the gradient. We can therefore express the gradient as a single matrix product.

To do so, we first collect the observed labels in the column

$$\mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_N \end{pmatrix} \in \mathbb{R}^{N \times 1}.$$

We also need to apply the sigmoid to all the scores in $\mathbf{X}\boldsymbol{\theta}$. For a column

$$\mathbf{s} = \begin{pmatrix} s_1 \\ \vdots \\ s_N \end{pmatrix} \in \mathbb{R}^{N \times 1},$$

we define

$$\boldsymbol{\sigma}(\mathbf{s}) := \begin{pmatrix} \sigma(s_1) \\ \vdots \\ \sigma(s_N) \end{pmatrix} \in (0, 1)^{N \times 1}.$$

Thus σ acts on one real number, while $\boldsymbol{\sigma}$ applies the same scalar sigmoid separately to every entry of a column.

Since $\mathbf{X}\boldsymbol{\theta}$ contains the scores of all the observations, the corresponding predicted probabilities can be collected as

$$\mathbf{p}(\boldsymbol{\theta}) := \begin{pmatrix} p_{\boldsymbol{\theta}}(\mathbf{x}_1) \\ \vdots \\ p_{\boldsymbol{\theta}}(\mathbf{x}_N) \end{pmatrix} = \boldsymbol{\sigma}(\mathbf{X}\boldsymbol{\theta}) \in (0, 1)^{N \times 1}. \quad (34)$$

Observe that the entries of this column refer to different observations, with each entry giving the probability assigned to class 1 for its own observation.

Now consider coordinate j of

$$\mathbf{X}^\top(\mathbf{p}(\boldsymbol{\theta}) - \mathbf{y}).$$

It is

$$\left[\mathbf{X}^\top(\mathbf{p}(\boldsymbol{\theta}) - \mathbf{y}) \right]_j = \sum_{i=1}^N X_{ij}(p_{\boldsymbol{\theta}}(\mathbf{x}_i) - y_i),$$

which is exactly the derivative in (33). Consequently,

$$\boxed{\nabla \mathcal{L}_{\log}(\boldsymbol{\theta}) = \mathbf{X}^\top(\mathbf{p}(\boldsymbol{\theta}) - \mathbf{y}) = \mathbf{X}^\top(\boldsymbol{\sigma}(\mathbf{X}\boldsymbol{\theta}) - \mathbf{y})}. \quad (35)$$

As a sanity check, notice that $\mathbf{X}^\top$ has shape $(d+1) \times N$, while the column $\mathbf{p}(\boldsymbol{\theta}) - \mathbf{y}$ has shape $N \times 1$. Therefore, as expected, their product has shape $(d+1) \times 1$: the gradient has one entry for each parameter, with the derivative with respect to the intercept appearing first.

Comparison with linear regression. Recall that, for the squared loss in linear regression, we obtained

$$\nabla \mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) = 2\mathbf{X}^\top(\mathbf{X}\boldsymbol{\theta} - \mathbf{y}).$$

For logistic loss, we have instead

$$\nabla \mathcal{L}_{\log}(\boldsymbol{\theta}) = \mathbf{X}^\top(\boldsymbol{\sigma}(\mathbf{X}\boldsymbol{\theta}) - \mathbf{y}).$$

The two gradients therefore have the same basic structure: $\mathbf{X}^\top$ multiplies a column of prediction-minus-target differences. In linear regression the predictions are $\mathbf{X}\boldsymbol{\theta}$, whereas here they are the probabilities $\boldsymbol{\sigma}(\mathbf{X}\boldsymbol{\theta})$. The factor 2 is also absent.

What does a stationary point tell us? Setting the gradient to zero finds the stationary points. Using (35), the condition is

$$\mathbf{X}^\top(\boldsymbol{\sigma}(\mathbf{X}\boldsymbol{\theta}) - \mathbf{y}) = \mathbf{0}. \quad (36)$$

Because the sigmoid is nonlinear, these are nonlinear equations in the parameters. Therefore, we cannot rearrange them into a linear system as we did with the normal equations for least squares, and in general there is no comparable explicit formula for the solution. However, we already have a formula for the gradient, so we can try to improve the parameters iteratively with gradient descent.

We remark that, if a stationary point exists, then it is a *global minimiser* of this objective. The interested and mathematically inclined reader can find a proof of this fact in Appendix A. The same appendix also explains why a finite minimiser need not exist: for some datasets, the loss keeps approaching zero as the parameters become arbitrarily large, even though the predicted labels are already correct.

2.9 Gradient descent

Recall from Lecture 2 that gradient descent subtracts the current gradient multiplied by the learning rate. Starting from $\boldsymbol{\theta}^{(0)}$ and choosing $\eta > 0$, substituting (35) gives

$$\boxed{\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta \mathbf{X}^\top (\boldsymbol{\sigma}(\mathbf{X}\boldsymbol{\theta}^{(t)}) - \mathbf{y}), \quad t = 0, 1, 2, \dots} \quad (37)$$

Fitting this probability model by minimising $\mathcal{L}_{\log}$ is called *logistic regression*. Equation (37) gives its gradient-descent update.

If we are also interested in keeping track of the intermediate quantities appearing in the update, we can write

$$\begin{aligned} \mathbf{s}^{(t)} &:= \mathbf{X}\boldsymbol{\theta}^{(t)} && \text{column of scores,} \\ \mathbf{p}^{(t)} &:= \boldsymbol{\sigma}(\mathbf{s}^{(t)}) && \text{column of predicted probabilities,} \\ \mathbf{r}^{(t)} &:= \mathbf{p}^{(t)} - \mathbf{y} && \text{column of prediction-minus-label differences,} \\ \boldsymbol{\theta}^{(t+1)} &:= \boldsymbol{\theta}^{(t)} - \eta \mathbf{X}^\top \mathbf{r}^{(t)} && \text{updated parameter column.} \end{aligned} \quad (38)$$

The first three columns have shape $N \times 1$, while $\boldsymbol{\theta}^{(t+1)}$ has shape $(d+1) \times 1$.

All coordinates are updated simultaneously using the probabilities computed at $\boldsymbol{\theta}^{(t)}$. Only after the whole parameter column has been updated do we recompute the scores, probabilities and prediction-minus-label differences for the next iteration. The inputs and observed labels remain fixed throughout.

This is the same computational pattern as in Lecture 2, with an extra sigmoid step before subtracting the targets.

2.10 Example: gradient descent with one input

We have derived the update for any number of features. To practise the calculations involved in this update, we now return to the seven observations in (15), where $d = 1$. The design

matrix has one row $(1, x_i)$ per observation, so in this example

$$\mathbf{X} = \begin{pmatrix} 1 & -2 \\ 1 & -1 \\ 1 & 0 \\ 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{pmatrix} \in \mathbb{R}^{7 \times 2}.$$

We use the general update (37), starting from

$$b^{(0)} = 0, \quad a^{(0)} = 0, \quad \eta = 0.2.$$

Every initial score is zero, so every probability is $1/2$. Our tie rule therefore predicts class 1 for every observation, which gives four mistakes. To compute the gradient, we subtract each observed label from its probability and then multiply by the input for the slope contribution:

i	1	2	3	4	5	6	7
x_i	-2	-1	0	1	2	3	4
y_i	0	0	0	1	1	0	1
$p_i^{(0)}$	1/2	1/2	1/2	1/2	1/2	1/2	1/2
$r_i^{(0)}$	1/2	1/2	1/2	-1/2	-1/2	1/2	-1/2
$x_i r_i^{(0)}$	-1	-1/2	0	-1/2	-1	3/2	-2

(39)

The first column of $\mathbf{X}$ consists of ones, while its second column contains the inputs. Therefore, the two entries of $\mathbf{X}^\top \mathbf{r}^{(0)}$ are the sums of the last two rows:

$$\begin{aligned} \frac{\partial \mathcal{L}_{\log}}{\partial b}(\boldsymbol{\theta}^{(0)}) &= \frac{1}{2} + \frac{1}{2} + \frac{1}{2} - \frac{1}{2} - \frac{1}{2} + \frac{1}{2} - \frac{1}{2} = \frac{1}{2}, \\ \frac{\partial \mathcal{L}_{\log}}{\partial a}(\boldsymbol{\theta}^{(0)}) &= -1 - \frac{1}{2} + 0 - \frac{1}{2} - 1 + \frac{3}{2} - 2 = -\frac{7}{2}. \end{aligned} \tag{40}$$

Thus the general matrix formula gives

$$\nabla \mathcal{L}_{\log}(\boldsymbol{\theta}^{(0)}) = \mathbf{X}^\top \mathbf{r}^{(0)} = \begin{pmatrix} 1/2 \\ -7/2 \end{pmatrix}.$$

Consequently, the first simultaneous update is

$$\begin{pmatrix} b^{(1)} \\ a^{(1)} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} - 0.2 \begin{pmatrix} 1/2 \\ -7/2 \end{pmatrix} = \begin{pmatrix} -0.1 \\ 0.7 \end{pmatrix}. \tag{41}$$

Both parameters have changed, so the new score is $f_{\boldsymbol{\theta}^{(1)}}(x) = -0.1 + 0.7x$. Substituting the seven inputs and applying the sigmoid gives

x_i	-2	-1	0	1	2	3	4
$s_i^{(1)}$	-1.5	-0.8	-0.1	0.6	1.3	2.0	2.7
$p_i^{(1)}$	0.1824	0.3100	0.4750	0.6457	0.7858	0.8808	0.9370
predicted label	0	0	0	1	1	1	1
y_i	0	0	0	1	1	0	1

(42)

The probabilities are rounded to four decimal places. Comparing the last two rows shows that only the observation at $x = 3$ is misclassified. Therefore, the first step reduces the error count from four to one, which is already the smallest possible count for a threshold on this dataset.

We can also check what happened to the objective that we actually optimised. Initially, every observation has probability $1/2$ for its observed label, so

$$\mathcal{L}_{\log}(\boldsymbol{\theta}^{(0)}) = 7 \log 2 \approx 4.85203.$$

After the update, we use the observed label to choose the appropriate score-loss formula from (22). This gives

$$\begin{aligned} \mathcal{L}_{\log}(\boldsymbol{\theta}^{(1)}) &= \log(1 + e^{-1.5}) + \log(1 + e^{-0.8}) + \log(1 + e^{-0.1}) \\ &\quad + \log(1 + e^{-0.6}) + \log(1 + e^{-1.3}) + \log(1 + e^2) \\ &\quad + \log(1 + e^{-2.7}) \\ &\approx 4.08738. \end{aligned}$$

In particular, the observation at $x = 3$ contributes $\log(1 + e^2)$, because its label is zero even though its score is positive. Therefore, this observation makes a substantial contribution to the total loss and continues to influence the updates, even though the classifier already achieves the smallest possible error count.

2.11 Continuing the same example

After the first step, we must recompute the scores and probabilities at the new parameters before taking another step. Repeating this procedure with the same learning rate gives the following values:

Iteration t	Intercept $b^{(t)}$	Slope $a^{(t)}$	$\mathcal{L}_{\log}(\boldsymbol{\theta}^{(t)})$	$\mathcal{E}(\boldsymbol{\theta}^{(t)})$
0	0.00000	0.00000	4.85203	4
1	-0.10000	0.70000	4.08738	1
2	-0.34336	0.51341	3.76388	1
5	-0.69675	0.60092	3.55828	2
20	-1.13979	0.74078	3.46285	2

These values were computed without rounding the intermediate iterates. The logistic loss decreases throughout the first twenty updates, but the error count does not follow the same pattern. In fact, the count first increases at $t = 4$, when the score at $x = 1$ becomes negative. Since that observation has label 1, the classifier now makes a second mistake.

To understand this change geometrically, recall that the threshold is $-b/a$ when $a > 0$. It is approximately 0.143 after the first step and 1.539 after twenty steps. Therefore, as the parameters change, the threshold moves past $x = 1$. Meanwhile, the probability assigned to class 1 at the incorrectly classified point $x = 3$ decreases, which improves its contribution to logistic loss. The algorithm balances all seven contributions, so it may accept an additional classification mistake while reducing their total loss.

The right-hand plot compares the error count with the rescaled logistic loss on the same axes. The rescaled curve stays above the error count, as guaranteed by (23), even where the two quantities move in opposite directions. The factor $1/\log 2$ is used only for this comparison, so it does not change any of the updates that we computed.

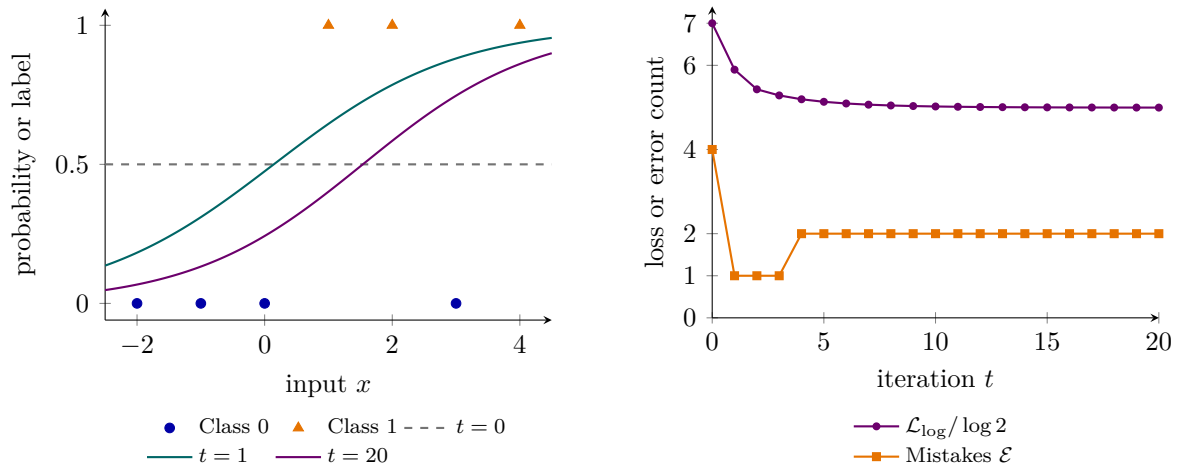


Figure 7: On the left, the probability model changes as gradient descent updates its parameters, while the observed labels stay fixed. On the right, the rescaled logistic loss decreases, but the error count rises from one to two at iteration four. Each marker gives a computed iterate.

We have therefore seen both the usefulness and the limitation of the surrogate. Its derivatives give us a practical fitting procedure, and a sufficiently small value bounds the number of mistakes. However, reducing it does not force every class prediction to improve.

3 Multiclass classification

So far, we have considered problems with two possible classes. However, the same input may need to be assigned to one of several classes.

For example, recognising a handwritten digit means choosing among the ten digits $0, 1, \dots, 9$, even though different images of the same digit can look quite different.

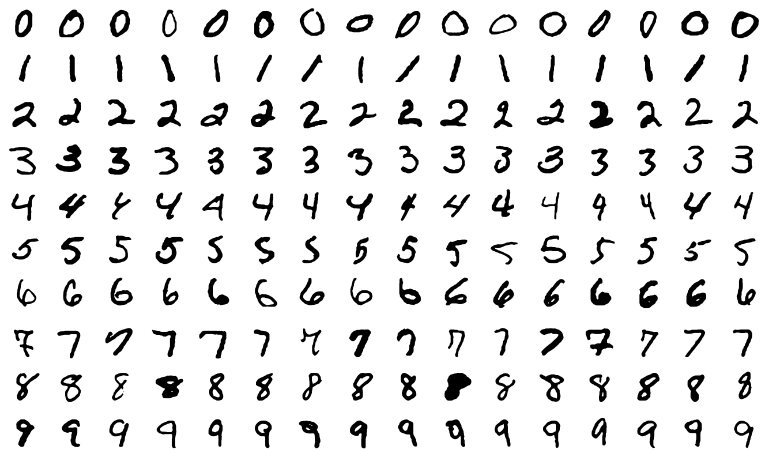


Figure 8: Handwritten digits form ten classes. The images within each row have the same label, but their shapes vary. The task is to predict the digit from the image, not to reproduce the image itself.

As before, we can represent an observation by an input row $\mathbf{x} \in \mathbb{R}^{1 \times d}$. We now want to predict one of K possible labels, which we write as

$$\{1, \dots, K\}.$$

In the digit example, these $K = 10$ class indices stand for the digits $0, \dots, 9$.

The basic construction is similar to binary classification. We first compute scores, then transform the scores into probabilities, and finally choose the class with the largest probability. The difference is that we now need one score and one probability for each class.

3.1 One affine score for each class

In binary classification, one affine score was transformed by the sigmoid into the probability of class 1. With K classes, we instead use one affine score, or *logit*, per class.

For class k , let

$$\mathbf{w}_k = \begin{pmatrix} w_{1k} \\ \vdots \\ w_{dk} \end{pmatrix} \in \mathbb{R}^{d \times 1},$$

and define

$$f_{\theta_k}(\mathbf{x}) = b_k + \mathbf{x}\mathbf{w}_k, \quad \boldsymbol{\theta}_k = \begin{pmatrix} b_k \\ \mathbf{w}_k \end{pmatrix} \in \mathbb{R}^{(d+1) \times 1}, \quad k = 1, \dots, K. \quad (43)$$

Putting these parameter columns side by side gives the parameter matrix

$$\boldsymbol{\Theta} = \begin{pmatrix} \boldsymbol{\theta}_1 & \dots & \boldsymbol{\theta}_K \end{pmatrix} \in \mathbb{R}^{(d+1) \times K}. \quad (44)$$

Thus the first row of $\boldsymbol{\Theta}$ contains the K intercepts, while each remaining row contains the weights of one input feature for all the classes.

For a single input $\mathbf{x}$, the K scores form the row

$$\mathbf{f}_{\boldsymbol{\Theta}}(\mathbf{x}) = \begin{pmatrix} f_{\theta_1}(\mathbf{x}) & \dots & f_{\theta_K}(\mathbf{x}) \end{pmatrix}.$$

We can also see how these scores are produced over a whole dataset. Using the same design matrix $\mathbf{X}$ as before,

$$\mathbf{X}\boldsymbol{\Theta} \in \mathbb{R}^{N \times K}$$

contains one row for each observation and one column for each class. In particular,

$$[\mathbf{X}\boldsymbol{\Theta}]_{ik} = f_{\theta_k}(\mathbf{x}_i).$$

Thus row i contains all the class scores for observation i .

3.2 Softmax turns the scores into probabilities

The scores can take any real values, so they cannot themselves be probabilities. Moreover, simply applying the sigmoid separately to all K scores would not work: although every resulting number would lie between zero and one, the K numbers would not necessarily sum to one.

Instead, we use the *softmax* transformation. For a score row

$$\mathbf{s} = (s_1, \dots, s_K),$$

softmax assigns to class k the probability

$$p_k(\mathbf{s}) := [\text{softmax}(\mathbf{s})]_k = \frac{e^{s_k}}{\sum_{r=1}^K e^{s_r}}, \quad k = 1, \dots, K. \quad (45)$$

It is useful to think of this transformation in three steps. Starting from the scores

$$(s_1, \dots, s_K),$$

we

(i) exponentiate every score, obtaining

$$(e^{s_1}, \dots, e^{s_K});$$

(ii) add these positive numbers,

$$\sum_{r=1}^K e^{s_r};$$

(iii) divide every exponentiated score by this same sum.

The resulting numbers are all positive. Moreover,

$$\sum_{k=1}^K p_k(\mathbf{s}) = \frac{\sum_{k=1}^K e^{s_k}}{\sum_{r=1}^K e^{s_r}} = 1,$$

so they form a probability distribution over the K classes.

Applying softmax to the affine score row gives our multiclass probability model:

$$\mathbf{p}_{\Theta}(\mathbf{x}) := \text{softmax}(\mathbf{f}_{\Theta}(\mathbf{x})). \quad (46)$$

Its k -th component is therefore

$$p_{\Theta,k}(\mathbf{x}) = \frac{\exp(f_{\theta_k}(\mathbf{x}))}{\sum_{r=1}^K \exp(f_{\theta_r}(\mathbf{x}))}.$$

An example with three classes. Suppose that, at some input, the score row is

$$\mathbf{s} = (0, \log 2, \log 3).$$

We first exponentiate the three scores:

$$(e^0, e^{\log 2}, e^{\log 3}) = (1, 2, 3).$$

We then add these numbers:

$$1 + 2 + 3 = 6.$$

Finally, we divide each number by this same sum:

$$\text{softmax}((0, \log 2, \log 3)) = \left(\frac{1}{6}, \frac{1}{3}, \frac{1}{2}\right).$$

Thus the probabilities assigned to classes 1, 2, 3 are respectively

$$\frac{1}{6}, \quad \frac{1}{3}, \quad \frac{1}{2}.$$

To obtain a class prediction, we simply choose a class with the largest probability. With the same tie-breaking convention as before, we write

$$h_{\mathbf{p}}(\mathbf{x}) = \max_{k \in \{1, \dots, K\}} \left(\text{argmax}_{k \in \{1, \dots, K\}} p_k(\mathbf{x}) \right).$$

In the example above, class 3 has the largest probability, so the classifier predicts class 3.

Notice that, unlike in binary classification, the largest probability does not need to exceed $1/2$. For example,

$$(0.40, 0.35, 0.25)$$

still leads us to predict class 1, simply because 0.40 is the largest of the three probabilities.

3.3 Cross-entropy loss

We now need an objective for fitting the multiclass model. As in binary classification, we use the probability assigned to the class that was actually observed.

For a probability row

$$\mathbf{p} = (p_1, \dots, p_K), \quad p_k > 0, \quad \sum_{k=1}^K p_k = 1,$$

and an observed class

$$y \in \{1, \dots, K\},$$

we define the *categorical cross-entropy loss*

$$\boxed{\ell_{\text{multi}}(\mathbf{p}, y) = -\log p_y.} \quad (47)$$

Thus the loss simply takes the probability assigned to the observed class and applies the negative logarithm. Assigning a large probability to the observed class gives a small loss, whereas assigning almost no probability to it gives a large loss.

For example, suppose that

$$\mathbf{p} = \left(\frac{1}{6}, \frac{1}{3}, \frac{1}{2}\right)$$

and the observed class is 2. Then

$$\ell_{\text{multi}}\left(\left(\frac{1}{6}, \frac{1}{3}, \frac{1}{2}\right), 2\right) = -\log(1/3) = \log 3.$$

There is also a useful way to write the loss directly in terms of the scores. For a single observation with score row

$$\mathbf{s} = (s_1, \dots, s_K)$$

and observed class y , substituting the softmax probability gives

$$\ell_{\text{multi}}(\text{softmax}(\mathbf{s}), y) = -\log\left(\frac{e^{s_y}}{\sum_{r=1}^K e^{s_r}}\right) = -s_y + \log\left(\sum_{r=1}^K e^{s_r}\right). \quad (48)$$

This expression gives the cross-entropy loss for a single observation. As in binary classification, we obtain an objective for fitting the model by adding the losses over all the observations:

$$\mathcal{L}_{\text{multi}}(\Theta) := \sum_{i=1}^N \ell_{\text{multi}}(\mathbf{p}_{\Theta}(\mathbf{x}_i), y_i) = -\sum_{i=1}^N \log\left([\mathbf{p}_{\Theta}(\mathbf{x}_i)]_{y_i}\right). \quad (49)$$

Here $[\mathbf{p}_{\Theta}(\mathbf{x}_i)]_{y_i}$ is the y_i -th component of the predicted probability row, namely the probability assigned to the observed class.

In principle, we could now differentiate this objective and use gradient descent exactly as we did in binary classification. The calculation is more involved because every observation has K scores whose softmax probabilities depend on one another. We will not derive these gradients here.

4 Exercises

The exercises follow the same path as the chapter. We first practise the sigmoid derivative in Exercise 1, and then compare logistic loss with the number of mistakes in Exercise 2. Exercise 3 works through one binary gradient-descent update, while Exercise 4 practises softmax and multiclass cross-entropy.

Throughout the exercises, use natural logarithms and sums of losses, without dividing by N , unless a mean is explicitly specified. In particular, remember the factor $1/\log 2$ when using (20). A calculator may be used for the final numerical values, but keep exact expressions until that final step.

- 1. Derivative of the logistic sigmoid.** Starting from $\sigma(s) = (1 + e^{-s})^{-1}$, use the chain rule to show that

$$\sigma'(s) = \sigma(s)\sigma(-s) = \sigma(s)(1 - \sigma(s)) \quad \text{for every } s \in \mathbb{R}.$$

To organise the calculation, first identify the inner and outer functions and find their derivatives. Then apply the chain rule, using (21) to obtain the final equality.

- 2. A smaller surrogate loss need not mean fewer mistakes.** Two observations both have label 1. One probability predictor assigns class 1 probabilities 0.51, 0.51, while another assigns 0.49, 0.99. In both cases, turn these probabilities into labels using the threshold rule $\text{step}(p - 1/2)$.

- Compute each classifier's predicted labels, number of mistakes, and sum of logistic losses $\ell_{\log}$.
- Using your calculations, determine which classifier is preferred by each objective. Then explain why their different preferences do not contradict (20).
- Now consider a dataset of N observations on which the mean logistic loss is at most 0.03. What error-rate bound follows from (20)? Using the same inequality, determine what you can conclude if the sum of logistic losses is strictly less than $\log 2$.

- 3. One binary gradient-descent update.** Consider the dataset

i	1	2	3
x_i	-1	0	2
y_i	0	1	1

and the probability model $p_{\theta}(x) = \sigma(b + ax)$, with $\theta = (b, a)^{\top}$. Start from

$$\theta^{(0)} = \begin{pmatrix} 0 \\ \log 3 \end{pmatrix}, \quad \eta = \frac{1}{5}.$$

- First construct $\mathbf{X}$, and then compute the score column $\mathbf{s}^{(0)}$, probability column $\mathbf{p}^{(0)}$, and differences $\mathbf{r}^{(0)} = \mathbf{p}^{(0)} - \mathbf{y}$. Keep these values exact.
- Using these differences, apply (35) to calculate the gradient. Then perform one simultaneous update using (37), giving both the new intercept and the new slope.
- Compute the predicted labels and empirical logistic loss before and after the update. Use the tie rule from (2). By comparing the two sets of predictions, determine whether the number of mistakes has changed. Then explain how your answer is consistent with the change in logistic loss.

4. Softmax and cross-entropy. Consider two observations with three classes. Their score rows are

$$\mathbf{s}_1 = (0, \log 2, \log 3), \quad \mathbf{s}_2 = (\log 4, 0, \log 2),$$

and their observed labels are

$$y_1 = 2, \quad y_2 = 1.$$

- (a) Apply softmax to each score row. For each observation, first exponentiate the scores, then add the exponentials, and finally divide each exponential by their sum. Keep the resulting probabilities exact.
- (b) Which class is predicted for each observation? State whether each prediction is correct.
- (c) Compute the categorical cross-entropy loss of each observation,

$$\ell_{\text{multi}}(\text{softmax}(\mathbf{s}_i), y_i),$$

and hence compute the summed loss over the two observations.

- (d) Verify the two individual losses again using the score form

$$\ell_{\text{multi}}(\text{softmax}(\mathbf{s}), y) = -s_y + \log \left(\sum_{k=1}^K e^{s_k} \right).$$

A Stationary points and finite minimisers in the binary case

The binary gradient tells us how to update the parameters, but it also lets us ask what happens when there is no update to make because the gradient is zero. To answer this question, we first show that every stationary point is a global minimiser. However, this does not tell us whether such a point exists, so we then examine an example in which there is no finite minimiser. Throughout this appendix, we use the general notation from Section 1, which also covers the model with just one input.

Why a stationary point is a global minimiser. Suppose that $\nabla \mathcal{L}_{\log}(\boldsymbol{\theta}^*) = \mathbf{0}$, and let $\mathbf{v}$ be any change to the parameter column. We want to compare the loss at $\boldsymbol{\theta}^*$ with the loss at $\boldsymbol{\theta}^* + \mathbf{v}$. To do this using a single-variable calculation, we follow the line between them and write

$$g(\alpha) = \mathcal{L}_{\log}(\boldsymbol{\theta}^* + \alpha \mathbf{v}), \quad s_i(\alpha) = (\mathbf{X} \boldsymbol{\theta}^*)_i + \alpha c_i, \quad c_i = (\mathbf{X} \mathbf{v})_i.$$

Since only the scalar α varies, we can differentiate along this line. For each observation, the chain rule gives

$$\begin{aligned} \frac{d}{d\alpha} \ell_{\log}(\sigma(s_i(\alpha)), y_i) &= \frac{\sigma(s_i(\alpha)) - y_i}{\sigma(s_i(\alpha))(1 - \sigma(s_i(\alpha)))} \sigma(s_i(\alpha))(1 - \sigma(s_i(\alpha))) c_i \\ &= (\sigma(s_i(\alpha)) - y_i) c_i. \end{aligned}$$

Summing over the observations and differentiating once more therefore gives

$$\begin{aligned} g'(\alpha) &= \sum_{i=1}^N (\sigma(s_i(\alpha)) - y_i) c_i, \\ g''(\alpha) &= \sum_{i=1}^N \sigma(s_i(\alpha))(1 - \sigma(s_i(\alpha))) c_i^2 \geq 0. \end{aligned}$$

Moreover, our assumption that the original gradient is zero gives $g'(0) = \mathbf{v}^\top \nabla \mathcal{L}_{\log}(\boldsymbol{\theta}^*) = 0$. Since $g'' \geq 0$, the derivative g' is nondecreasing. Together with $g'(0) = 0$, this means that g' is nonnegative between zero and one. Therefore

$$\mathcal{L}_{\log}(\boldsymbol{\theta}^* + \mathbf{v}) = g(1) \geq g(0) = \mathcal{L}_{\log}(\boldsymbol{\theta}^*).$$

Every other parameter choice has this form, so $\boldsymbol{\theta}^*$ is a global minimiser. However, the argument began by assuming that a stationary point was available, and therefore does not establish that one exists.

A finite minimiser need not exist. To see why a finite minimiser need not exist, consider a separate dataset with one input feature:

i	1	2	3	4
x_i	-2	-1	1	2
y_i	0	0	1	1

(50)

Unlike the seven-point example in the main text, these observations can all be classified correctly by a single threshold. Indeed, if we set $\boldsymbol{\theta} = (0, a)^\top$, then the score is ax , and every $a > 0$ gives the correct predicted labels $(0, 0, 1, 1)$. However, the logistic loss still depends on how large a is, because

$$\mathcal{L}_{\log}((0, a)^\top) = 2 \log(1 + e^{-2a}) + 2 \log(1 + e^{-a}). \quad (51)$$

This value is positive for every finite a , but approaches zero as $a \rightarrow +\infty$. On the other hand, no finite affine parameter choice can achieve zero loss, because a finite sigmoid score never assigns probability exactly one to an observed label. Therefore the infimum is zero and is not attained.

The reason is that increasing a makes the predicted probabilities more confident without changing any predicted label. Thus the logistic loss keeps improving even though these classifiers already make no mistakes, and there is no finite parameter value at which it reaches its smallest possible value.

B An optional interpretation of the score: log-odds

The sigmoid turns a score into a probability, but we can also reverse this relation. To see how, let $p = \sigma(s)$. Then

$$p = \frac{e^s}{1 + e^s}, \quad 1 - p = \frac{1}{1 + e^s},$$

so dividing the first expression by the second gives

$$\frac{p}{1 - p} = e^s.$$

The ratio $p/(1 - p)$ is called the *odds* of class 1, and its logarithm is called the *log-odds*. Therefore, taking logarithms shows that the score is exactly the log-odds. In particular, for the general affine model,

$$\log\left(\frac{p_{\theta}(\mathbf{x})}{1 - p_{\theta}(\mathbf{x})}\right) = f_{\theta}(\mathbf{x}) = b + \mathbf{x}\mathbf{w}. \quad (52)$$

This gives another way to describe the model. Although the probability is not affine in the input, its log-odds are affine. For example, $p = 1/2$ gives odds equal to one and hence score zero, in agreement with the classification threshold used throughout the chapter.